

ASTRO RAIDER

Perk System — Game Design Document

1. Perk System Overview

Das Perk System ist das Herzstück des Progression Loops in Astro Raider. Perks werden durch Crystals freigeschaltet, die am Generator gegen Perk-Auswahlen eingetauscht werden. Jeder Perk hat bis zu 6 Level (Ults: 3 Level) und gehört einer Kategorie an.

1.1 Perk Kategorien

Kategorie	Max Slots	Taste	Max Level	Besonderheit
Passive	Unbegrenzt	—	6	Immer aktiv nach Freischaltung
Activation	3 (Q / E / C)	Q, E, C	6	Muss einem Slot zugewiesen werden
Ult	1 (X)	X	3	Mächtigste Kategorie — seltenste

1.2 Rarity System

Jeder Perk hat ein rarity Array mit 6 Werten (Level 1–6). Höherer Wert = häufiger im Pool. Die Gewichtung wird in PerkSelection.build_offer() per weighted random berechnet.

- Level 1 rarity ist die Basis-Erscheinungsrate
- Mit jedem Level sinkt die rarity (Perk wird seltener angeboten)
- Typische Werte: [10.0, 8.0, 6.0, 4.0, 2.0, 1.0]

1.3 Crystal Kosten & Leveling

Crystals werden am Generator gesammelt und gegen Perk-Selektionen eingetauscht. Die Kosten steigen exponentiell mit dem Player Level.

```
func exp_to_next() -> int:
    return ceil(25 + 12 * (current_player_level - 1) * 0.8)
```

Player Level	Crystals benötigt
1	10
2	35

2. Code Architektur

2.1 Kern-Scripts

Script	Verantwortlichkeit
PerkBuild.gd	Basisklasse für alle Perks. Enthält Level, has_unlocked, activate_perk(), get_value(), get_cooldown(), get_duration()
Perk.gd (Resource)	Data Container: image, rarity, value, cooldown, duration, description Arrays (je 6 Werte für Passive/Activation, 3 für Ult)
PerkData.gd	Registry: Keys enum, Keys_scene und Keys_res Dictionaries, statischer Cache für Scenes und Resources
PerkManager.gd	Coordinator: try_open_selector(), finalize_after_level_up(), Crystal-Kosten-Berechnung, Generator-Interaktion
PerkSelection.gd	Pool-Management: build_offer(), _get_available_perks(), Cooldown-System, prune_maxed()
PerkSlots.gd	Slot-Verwaltung: Q/E/C/X Slots, assign_to_slot(), activate_slot(), cooldown_started Signal Connection
ability_slot.gd	UI-Node: ProgressBar Cooldown, show_active(), start_cooldown(), _counting Flag

2.2 Stats Modifier System

Perks modifizieren Stats über ein Modifier-Dictionary statt direkter Addition. Mehrere Perks überschreiben sich dadurch nie gegenseitig.

```
stats.set_modifier(stats.max_speed_modifiers, Key, bonus_value)
stats.remove_modifier(stats.max_speed_modifiers, Key)
```

Getter berechnen den Gesamtwert dynamisch aus allen aktiven Modifiern. Immer get_*_total() nutzen — nie die Basis-Variable direkt.

2.3 Perk Lifecycle

Schritt	Aktion
1	Spieler drückt F am Generator → PerkManager.try_open_selector()
2	PerkSelection.build_offer() filtert verfügbare Perks, wählt 3 per rarity-Gewichtung
3	Spieler wählt Perk → select_by_index() → perk.level_up_perk()
4	PerkManager.finalize_after_level_up() — bei Activation: Slot-Auswahl UI öffnet sich
5	PerkSlots.assign_to_slot() verbindet cooldown_started Signal mit ability_slot.start_cooldown()
6	Spieler drückt Q/E/C/X → PerkSlots.activate_slot() → perk.activate_perk()

2.4 Cooldown System

Das Cooldown System verwendet ability_slot als Single Source of Truth.

- show_active(max_cooldown) — setzt Bar auf voll, _counting = false (Shield aktiv)
- start_cooldown(duration) — setzt value auf duration, _counting = true (zählt runter)
- is_on_cooldown() in PerkBuild prüft ability_slot_ref.cooldown.value > 0.0
- Kein separater Timer — ProgressBar ist die einzige Wahrheit

3. Neuen Perk erstellen — Schritt für Schritt

Dieses Kapitel erklärt vollständig wie ein neuer Perk von Grund auf erstellt wird. Jeder Perk braucht: ein Icon, eine Resource (.tres), eine Scene (.tscn) mit Script, einen Eintrag in PerkData.gd und einen Node unter dem Player.

3.1 Schritt 1 — Icon vorbereiten

Erstelle ein quadratisches Icon-PNG (empfohlen: 64x64 oder 128x128 Pixel) im Pixel Art Stil passend zum Game. Das Icon wird in der Perk Resource als image: Texture2D hinterlegt und im UI angezeigt.

- Dateiname: z.B. perk_icon_my_perk.png
- Ordner: res://Perks/Icons/ (oder entsprechender Asset-Ordner)
- Format: PNG mit Transparenz (RGBA)

3.2 Schritt 2 — Perk Resource erstellen (.tres)

Erstelle eine neue Resource vom Typ Perk (rechtsklick im FileSystem → New Resource → Perk). Speichere sie unter:

```
res://Perks/Resources/Perk_MeinPerk.tres
```

General

Feld	Beschreibung
image	Dein Icon als Texture2D
Key	Wähle den passenden Key aus PerkData.Keys (muss vorher eingetragen sein)
perk_name	Anzeigename im UI, z.B. "My Perk"
description	Einzel-String mit Platzhaltern: {{value}}, {{cooldown}}, {{duration}}
type	Type_keys: Movement / Defens / Offens / Team / Mining
active_type	Active_type_keys — bestimmt ob Passive, Activation, Ult etc.

active_type — Werte

Wert	Verwendung
Always	Passive Perks — activate_perk() wird jeden Frame in _process() aufgerufen
Activation	Activation Perks — Spieler drückt Q/E/C manuell
Ult	Ult Perks — Spieler drückt X, max Level = 3 statt 6
Start	Einmalig beim Freischalten aktiviert
OneTime	Einmalig aktivierbar, danach inaktiv
TimeDelay	Wird nach einer Verzögerung automatisch aktiviert
Custom_Condition	Eigene Bedingung im Script prüfen

Classification

Feld	Beschreibung
tags	Array[Tag_keys]: Aggro / Sniper / Tank / Miner / Controller / Mobility / Sustain
mechanics	Array[Mechanic_keys]: Projectile / Melee / Crit / Heal / Dash / Shield / Crystal / Mining / Stun
required_perks	Keys die bereits vorhanden sein müssen damit dieser Perk erscheint
excluded_perks	Keys die nicht gleichzeitig aktiv sein dürfen

Scaling Arrays — WICHTIG: Sync Array Sizes drücken!

Nach dem Setzen von `active_type` immer den Button "Sync Array Sizes" im Inspector drücken. Er passt alle Arrays auf die richtige Größe an (6 für alle, 3 für Ult).

Array	Bedeutung
rarity	Erscheinungsgewicht pro Level — typisch [10.0, 8.0, 6.0, 4.0, 2.0, 1.0]
value	Hauptwert des Perks pro Level — z.B. Schaden, Anzahl, Bonus
cooldown	Cooldown in Sekunden pro Level (0 für Passive)
duration	Effekt-Dauer in Sekunden pro Level (0 wenn nicht relevant)
cost	Crystal-Kosten pro Level (wird vom Economy-System verwendet)

description Platzhalter

Die description ist ein einzelner String. Platzhalter werden zur Laufzeit durch den Level-Wert ersetzt:

Platzhalter	Ersetzt durch
{{value}}	<code>perk_res.value[level - 1]</code>
{{cooldown}}	<code>perk_res.cooldown[level - 1]</code> (formatiert ohne unnötige Dezimalstellen)
{{duration}}	<code>perk_res.duration[level - 1]</code> (formatiert ohne unnötige Dezimalstellen)

Beispiel: `"Summons {{value}} claws for {{duration}}s. Cooldown: {{cooldown}}s."`

3.3 Schritt 3 — Key in PerkData.gd eintragen

Öffne `PerkData.gd` und füge den neuen Key an drei Stellen hinzu:

```
# 1) In Keys enum:  
enum Keys { ..., My_New_Perk, }
```

```
# 2) In Keys_scene:  
Keys.My_New_Perk: "res://Perks/PerkBuild/Perk_My_New_Perk.tscn",
```

```
# 3) In Keys_res:  
Keys.My_New_Perk: "res://Perks/Resources/Perk_My_New_Perk.tres",
```

⚠ Alle drei Einträge müssen exakt übereinstimmen. Fehlende Pfade führen zu `null>Returns` in `load_perk_res()` und `load_perk_scene()`.

3.4 Schritt 4 — PerkBuild Script erstellen

Erstelle ein neues GDScript das PerkBuild extended. Die Basisklasse stellt folgende Variablen und Methoden bereit die du nutzen sollst:

Variable / Methode	Typ	Beschreibung
player	Player	Referenz auf den Spieler-Node (via @export gesetzt)
stats	Stats	player.stats — Zugriff auf alle Stat-Getter
perk_res	Perk	Die geladene Resource für diesen Perk
Level	int	Aktuelles Level (1–6 oder 1–3 für Ult)
has_unlocked	bool	true wenn der Perk bereits freigeschaltet wurde
ability_slot_ref	ability_slot	UI-Slot Referenz — für Cooldown-Anzeige
get_value()	int	perk_res.value[Level - 1]
get_cooldown()	float	perk_res.cooldown[Level - 1]
get_duration()	float	perk_res.duration[Level - 1]
is_on_cooldown()	bool	Prüft ob ability_slot_ref noch Cooldown hat
get_player_res()	PlayerResource	Gibt die PlayerResource für diesen Spieler zurück
cooldown_started	Signal	emit(duration) um den Cooldown zu starten

Passive Perk — Template

active_type = Always. activate_perk() wird jeden Frame aufgerufen. Stats werden über set_modifier / remove_modifier gesetzt — nie direkte Addition.

```
extends PerkBuild

## Applies speed bonus modifier each frame while unlocked
func activate_perk() -> void:
    stats.set_modifier(stats.max_speed_modifiers, Key, float(get_value()))

## Removes modifier when perk is reset or not unlocked
func _reset_stats() -> void:
    stats.remove_modifier(stats.max_speed_modifiers, Key)
```

Activation Perk — Template

active_type = Activation. activate_perk() wird nur bei Tastendruck via PerkSlots aufgerufen. Cooldown wird mit cooldown_started.emit() gestartet.

```
extends PerkBuild

var _is_active := false
var _timer: Timer = null

func _ready() -> void:
    super()
```

```

        _timer = Timer.new()
        _timer.one_shot = true
        _timer.timeout.connect(_on_ended)
        add_child(_timer)

func activate_perk() -> void:
    if not has_unlocked or _is_active or is_on_cooldown():
        return
    _is_active = true
    _timer.start(get_duration())
    if is_instance_valid(ability_slot_ref):
        ability_slot_ref.show_active(get_cooldown())

func _on_ended() -> void:
    _is_active = false
    cooldown_started.emit(get_cooldown())

func _reset_stats() -> void:
    _is_active = false
    if is_instance_valid(_timer): _timer.stop()

```

Ult Perk — Template

active_type = Ult. Funktioniert wie ein Activation Perk aber mit max Level 3. Arrays in der Resource haben nur 3 Einträge. Taste X statt Q/E/C.

```

# Identisch zum Activation Template – nur Resource-Arrays haben 3 statt 6 Werte
# Ult Level 1 entspricht in Power ca. Passive Level 6
# Ult Level 3 entspricht in Power ca. Passive Level 18

```

3.5 Schritt 5 — Scene erstellen (.tscn)

Erstelle eine neue Scene mit einem Node als Root. Weise dem Root-Node das neue Script zu.

Scene Root	Script
Node (für Passive)	Perk_My_New_Perk.gd
Node (für Activation/Ult)	Perk_My_New_Perk.gd

⚠ Im Inspector der Scene: Key auf den neuen PerkData.Keys-Eintrag setzen. player muss via @export auf den Player-Node zeigen — das wird vom PerkManager beim Instanzieren gesetzt.

3.6 Schritt 6 — Node unter dem Player hinzufügen

Öffne die Player Scene. Navigiere zu Player → PerkManager → Perks. Instanziiere die neue Perk-Scene als Child-Node unter Perks. Weise im Inspector zu:

- Key → den neuen PerkData.Keys Eintrag
- player → den Player-Node (per Drag & Drop)
- Level → 1 (Startwert)
- has_unlocked → false (wird durch PerkManager gesetzt)

4. Implementierte Passive Perks

Perk	Effekt	Typ	Stats genutzt
Speed It Up	Erhöht max_speed und gravity_strength	Movement	max_speed_modifiers, gravity_strength_modifiers
Construction Expert	Erhöht bohrer_damage	Mining	bohrer_damage_modifiers
Coin Master	Spawnt Bonus Crystals beim Sammeln	Mining	—
Vampire Bite	Heilt bei Gegner-Kill (HP pro Kill = value)	Defens	current_health
Each Round Heal	Heilt % Max HP am Wave-Ende	Defens	get_max_hp_total()
Resilience	Reduziert Unverwundbarkeits-Frame Dauer	Defens	invincibility_frame_modifiers
Extra Health	Erhöht max_hp	Defens	max_hp_modifiers
Aim Bot	Projektile folgen automatisch Gegnern	Offens	—
Piercing Shot	Projektile durchdringen Gegner (hp += 1)	Offens	projectile_lives_modifiers
Power Shot	Erhöht Projektil-Schaden	Offens	projectile_damage_modifiers
Stun Grenade	Treffer aktivieren Stun auf Gegnern	Offens	has_stun_active, stun_strength
Critical Edge	Erhöht Crit Chance	Offens	crit_chance_modifiers
Emergency Heal	Auto-Heal wenn HP unter 25%	Defens	current_health, get_max_hp_total()
Anti Mine Detection	Zeigt Items hinter Wänden im Radius	Mining	PlayerRevealDetection.reveal_radius_tiles
Vampire Bite	Heilt bei Gegner-Kill	Defens	current_health

Synergien

Kombination	Effekt
Aim Bot + Piercing Shot	Projektile zielen automatisch und durchdringen mehrere Gegner
Aim Bot + Power Shot + Critical Edge	Hochgeschwindigkeits-Crit-Build
Vampire Bite + Each Round Heal + Emergency Heal	Maximale Sustain
Speed It Up + Resilience	Aggressiver Hit-and-Run Stil
Construction Expert + Coin Master	Mining-Build: mehr Crystals = mehr Selektionen
Stun Grenade + Aim Bot	Projektile treffen automatisch und stunnen

5. Implementierte Activation Perks

5.1 Barrier Shield

Eigenschaft	Details
Script	Perk_Barrier_Shield.gd
Effekt	Spawnt einen Shield um den Spieler. can_take_damage = false solange aktiv. 50% Knockback-Reduktion
HP pro Level	25 / 50 / 75 / 100 / 125 / 150
Cooldown	50 / 44 / 38 / 32 / 26 / 20 Sekunden — startet nach Shield-Zerstörung
Cooldown Start	Erst wenn Shield zerstört wird, nicht beim Aktivieren (show_active statt cooldown_started)
Visuell	Hexagon-Pattern Shader mit Glow-Effekt, hit_flash bei Treffer
Signal	PERK_barrier_shield_destroyed → _on_shield_destroyed()

5.2 Energy Overload

Eigenschaft	Details
Script	perk_energy_overload.gd
Effekt	Speed + Bohrer-Schaden Boost + schnelleres Schießen. Spieler wird via LUT Shader recolorisiert
Speed Bonus/Level	40 / 55 / 70 / 85 / 100 / 120
Dauer	Aus duration Array der Perk Resource
Cooldown	Startet nach Ablauf der Dauer
Visuell	LUT PNG Recolor (Orange/Rot), Vignette Screen Effekt via ScreenEffects Autoload
Shader	2-Zeilen LUT Textur: Zeile 1 = Original, Zeile 2 = Overload Farben
Attack Speed	attack_speed_modifiers wird während Aktivierung gesetzt

5.3 Impulse Drive (Dash)

Eigenschaft	Details
Script	Perk_Impulse_Drive.gd
Effekt	Dash in Input-Richtung (Fallback: last_move_direction). can_take_damage = false während Dash
Distanz/Level	80 / 95 / 110 / 125 / 140 / 160 Pixel (value Array)
Dauer	0.12s konstant alle Level
Cooldown	10 / 9 / 8 / 7 / 6 / 5 Sekunden
Tiles zerstören	1–6 Tile-Layer via GSignals.ENV_destroy_tile (tiles_destroyable_per_level)
Kill-Reduktion	Pro Kill während Dash: 25% Cooldown-Reduktion (0.3s Kill-Window nach Dash-Ende)
Kollision	move_and_collide — Wand-Hit stoppt Dash oder zerstört Tile

5.4 Blood Claws

Eigenschaft	Details
Script	perk_blood_claws.gd + claw.gd
Effekt	Spawnt organische Blut-Krallen die automatisch Gegner stechen und Tiles zerstören
Claw-Anzahl/Level	2 / 3 / 4 / 5 / 6 / 8 (value Array der Resource)
Reach/Level	55 / 58 / 62 / 66 / 70 / 75 Pixel (reach_per_level @export in PerkBuild)
Lunge Reach/Level	40 / 45 / 50 / 55 / 60 / 70 Pixel (lunge_reach_per_level @export)
Schaden/Level	8 / 11 / 14 / 18 / 22 / 28 (strike_damage_per_level @export)
Dauer	5.0 / 5.5 / 6.0 / 6.5 / 7.0 / 8.0 Sekunden (duration Array)
Cooldown	32 / 28 / 24 / 20 / 16 / 12 Sekunden (cooldown Array)
Despawn	Animiert — Krallen ziehen sich zum Spieler zurück, dann queue_free()
Spawn	Claws Root in get_tree().current_scene (nicht als Player Child — verhindert Rotation)
Angriff	Slingshot: kurzes Zurückziehen → schneller Lunge → Rückzug
Targeting	Gegner haben Priorität über Environment Tiles
Env-Zerstörung	Via GSignals.ENV_destroy_tile mit Linie aus Punkten (ENV_KILL_DAMAGE = 999999)

6. Implementierte Ult Perks

6.1 Temporal Collapse

Eigenschaft	Details
Script	perk_temporal_collapse.gd
Effekt	Verlangsamt alle Gegner für die Dauer via EnemyStats.time_warp_multiplier
Level	Max 3 (Ult-Regel: Arrays haben 3 Einträge)
Visuell	Screen-Shader Effekt + Uhr-Effekt
Cooldown	Startet nach Ablauf der Dauer

7. Geplante Perks (Nicht implementiert)

7.1 Geplante Activation Perks

Perk	Effekt	Cooldown L1/L6	Priorität
EMP	Stunnt alle Gegner im Umkreis	30s / 15s	Hoch
Gravity Well	Zieht alle Gegner zu einem Punkt	40s / 20s	Hoch
Fireball	Feuerball mit Burn-DoT	15s / 5s	Mittel
Void Claws	Klauen-Angriff: zieht Gegner heran + Schaden	20s / 10s	Mittel
Chain Lightning	Blitz springt zwischen Gegnern	18s / 8s	Mittel
Decoy	Köder der Gegner ablenkt	35s / 20s	Niedrig

7.2 Geplante Ult Perks

Ult	Effekt	Cooldown L1/L3	Priorität
Black Hole	Saugt alle Gegner an + kontinuierlicher Schaden	120s / 80s	Hoch
Time Warp	Verlangsamt alle Gegner + Screen-Shader	90s / 60s	Hoch
Adrenaline	Unverwundbar + 3x Schaden kurze Zeit	100s / 70s	Mittel

7.3 Geplante Passive Perks

Perk	Effekt	Synergie mit
Phantom	Erhöhte Speed während iFrame	Resilience, Speed It Up
Lucky Strike	Kills haben Chance Crystal zu dropen	Vampire Bite, Coin Master
Bounty Hunter	Bohrer-Kills dropen mehr Ressourcen	Construction Expert, Coin Master

8. Signal Referenz

8.1 Perk-relevante GSignals

Signal	Emittiert von	Empfangen von
PERK_barrier_shield_destroyed	BarrierShield (bei HP <= 0)	Perk_Barrier_Shield._on_shield_destroyed()
PERK_Aim_bot_activate(player, value)	Perk_Aim_Bot._on_player_shoot()	EnemyDetectorArea.activate_aim_bot()
PERK_event_collect_crystal(pos)	ItemCrystal.collect()	Zukünftig: Demon Commentator
PERK_show_items_behind_wall(pos)	Anti Mine Detection Perk	Enviroment._on_show_items_behind_wall()
PLA_is_shooting(player)	Player Shooting Logic	Perk_Aim_Bot._on_player_shoot()
ENE_killed_by(killer)	EnemyBaseTemplate.death()	Impulse Drive._on_enemy_killed()
ENV_destroy_tile(pos, damage)	Impulse Drive, Blood Claws	Enviroment._on_destroy_tile()
HIT_take_Damage(entity, dmg, crit)	Hitbox, Impulse Drive, Blood Claws	EnemyBaseTemplate.applay_damage()

8.2 PerkBuild Signals

Signal	Verwendung
cooldown_started(duration: float)	Wird in Perk-Subklassen emittiert. PerkSlots verbindet es mit ability_slot.start_cooldown() beim assign_to_slot()

9. Zukunftspläne

9.1 Synergy Tag System

Zukünftig sollen Perks Tags bekommen die bei Kombination die Erscheinungsrare anderer Perks erhöhen.

```
var tag_bonus := _calculate_tag_bonus(p)  # +30% pro bereits besessenen Tag-Perk
var effective_rarity := p.perk_res.get_rarity(p.Level) * (1.0 + tag_bonus)
```

Tag	Perks	Build-Stil
aggro	Vampire Bite, Impulse Drive, Blood Claws, Power Shot	Nahkampf, Kill-basierte Sustain
sniper	Aim Bot, Piercing Shot, Critical Edge, Power Shot	Projektil-fokussiert, Einzelziel
tank	Barrier Shield, Extra Health, Each Round Heal, Resilience	Defensiv, Generator-Schutz
miner	Construction Expert, Coin Master, Anti Mine Detection, Speed It Up	Ressourcen, schnelles Leveln
control	EMP, Gravity Well, Stun Grenade, Chain Lightning	Crowd Control, Gegner-Kontrolle

9.2 Perk Selector Kategorien

Kategorie	Beschreibung
Standard (aktuell)	3 zufällige Perks aus dem Pool, rarity-gewichtet
Activation Only	Nur Activation Perks — wenn alle 3 Slots leer oder gezielt getriggert
Ult Choice	Nur die 3 verfügbaren Ults — beim ersten Ult-Freischalten
Rare Pool	Nur Perks mit rarity < 3 — garantiert seltene Optionen
Synergy Offer	Bevorzugt Perks die bereits besessene Tags teilen

10. Aktueller Stand & Bekannte Probleme

Implementiert und funktional

- Rarity-gewichtete Perk-Auswahl
- Cooldown System via ProgressBar als Single Source of Truth
- Slot Assignment UI mit Pulse-Effekt
- Barrier Shield mit Hexagon-Shader und Cooldown nach Zerstörung
- Energy Overload mit LUT Recolor, Vignette Screen-Effekt und Attack Speed Boost
- Impulse Drive mit Tile-Zerstörung und Kill-basierter Cooldown-Reduktion
- Blood Claws mit organischer Tentakel-Bewegung, Slingshot-Angriff, Env-Zerstörung und animiertem Despawn
- Temporal Collapse — erster Ult implementiert
- Activation Perks können nach dem ersten Auswählen weiter aufgelevelt werden
- `can_take_damage` Check in `Hitbox.get_hit()` — Shield und Dash schützen vor Bomben

Offene Punkte

- Synergy Tag System noch nicht implementiert
- Perk Selector Kategorien noch nicht implementiert
- 6 geplante Activation Perks noch nicht gebaut
- 2 geplante Ult Perks (Black Hole, Adrenaline) noch nicht implementiert
- 3 passive Team-Perks (Multiplayer, Post-v1.0) noch nicht gebaut
- Demon Commentator Trigger für Perk-Events noch nicht verbunden
- Meta-Progression (Perks zwischen Runs freischalten) noch nicht implementiert